

PHASE 3 – Candidate Module & Job Applications

1. Phase Objective

Phase 3 introduces the **candidate side of the system**. After recruiters can post jobs (Phase 2), candidates must be able to **browse jobs, apply to jobs, and track application status**.

The focus of this phase is to implement a **complete and realistic candidate workflow** using Django ORM and Django templates, while keeping the system extensible for future AI-based evaluation.

2. Prerequisites (Completed in Earlier Phases)

Before starting Phase 3, the following components were already implemented:

From Phase 1

- Django project setup
- MySQL database connection
- Custom User model
- User roles (Candidate / Recruiter)
- Authentication and authorization

From Phase 2

- Job model
- Recruiter job posting flow
- Job status handling (active / closed)
- Bootstrap-based UI foundation

Phase 3 **does not modify** earlier logic; it only extends the system.

3. Technologies Used in Phase 3

Backend

- Django
- Django ORM
- MySQL

Frontend

- Django Templates
- Bootstrap 5 (via CDN)

No new Python packages were installed in this phase.

4. Application Model Design

4.1 Why an Application Model is Required

A **Job** represents a vacancy, but it does not track who applied. The **Application** model represents the relationship between a **Candidate** and a **Job**.

This model is essential for: - Preventing duplicate applications - Tracking application status - Future ranking and cutoff logic - Resume parsing and AI matching (Phase 4)

4.2 Application Model Fields

Field	Description
candidate	Candidate who applied
job	Job applied to
status	Current application status
score	Evaluation score (future use)
applied_at	Timestamp of application

4.3 Application Status Design

Application status is implemented using **choices**: - applied (default) - shortlisted - rejected - selected

This mirrors real-world recruitment pipelines.

5. Application Model Implementation

File: `accounts/models.py`

```
from django.conf import settings
from django.db import models
from django.utils import timezone

class Application(models.Model):
    STATUS_CHOICES=(
        ('applied', 'Applied'),
        ('shortlisted', 'Shortlisted'),
        ('rejected', 'Rejected'),
        ('selected', 'Selected'),
    )
```

```

candidate=models.ForeignKey(settings.AUTH_USER_MODEL,on_delete=models.CASCADE,related_name='ap

job=models.ForeignKey('Job',on_delete=models.CASCADE,related_name='applications')

status=models.CharField(max_length=20,choices=STATUS_CHOICES,default='applied')
score=models.IntegerField(null=True,blank=True)
applied_at=models.DateTimeField(default=timezone.now)

class Meta:
    unique_together=('candidate','job')

def __str__(self):
    return f'{self.candidate.email} - {self.job.title}'

```

Design Notes

- `unique_together` prevents multiple applications to the same job
- `score` is nullable for future AI evaluation

5.1 Database Migration Commands

After adding the Application model:

```

python manage.py makemigrations
python manage.py migrate

```

This created the `application` table in MySQL with foreign-key relationships.

6. Candidate Job Listing

6.1 Design Decision

Candidates should see **all active jobs** without filtering or ranking at this stage.

Only jobs with:

```
status = 'active'
```

are displayed.

6.2 Candidate Job List View

File: `accounts/views.py`

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def candidate_job_list(request):
    jobs=Job.objects.filter(status='active')
    return render(request, 'accounts/candidate_job_list.html',{'jobs':jobs})
```

6.3 Candidate Job List Template

File: `accounts/templates/accounts/candidate_job_list.html`

Key UI features: - Bootstrap cards - Job title, company, location, experience - Skills display - Apply button

The Apply button is enabled only after application logic is implemented.

6.4 Candidate Job List URL

File: `accounts/urls.py`

```
path('jobs/', candidate_job_list, name='candidate_job_list'),
```

7. Apply to Job Flow

7.1 Apply Flow Logic

When a candidate clicks Apply: 1. Job must be active 2. Candidate must be logged in 3. Duplicate applications are checked 4. Application record is created 5. Status is set to `applied`

7.2 Apply View Implementation

File: `accounts/views.py`

```
from django.shortcuts import get_object_or_404, redirect
from django.contrib import messages

@login_required
def apply_to_job(request, job_id):
    job=get_object_or_404(Job, id=job_id, status='active')

    if Application.objects.filter(candidate=request.user, job=job).exists():
        messages.warning(request, 'You have already applied to this job')
```

```
return redirect('candidate_job_list')
```

```
Application.objects.create(candidate=request.user, job=job)
messages.success(request, 'Application submitted successfully')
return redirect('candidate_job_list')
```

7.3 Apply URL

File: `accounts/urls.py`

```
path('jobs/apply/<int:job_id>/', apply_to_job, name='apply_to_job'),
```

8. Candidate Dashboard

8.1 Purpose

The candidate dashboard allows candidates to **track all their applications** and view current statuses.

8.2 Candidate Dashboard View

File: `accounts/views.py`

```
@login_required
def candidate_dashboard(request):

    applications=Application.objects.filter(candidate=request.user).select_related('job')
    return render(request, 'accounts/candidate_dashboard.html',
        {'applications':applications})
```

8.3 Candidate Dashboard Template

File: `accounts/templates/accounts/candidate_dashboard.html`

UI elements: - Bootstrap table - Status badges - Job and company information - Applied date

8.4 Candidate Dashboard URL

File: `accounts/urls.py`

```
path('candidate/dashboard/', candidate_dashboard, name='candidate_dashboard'),
```

9. Candidate UI Flow (End-to-End)

1. Candidate logs in
 2. Candidate views available jobs
 3. Candidate applies to a job
 4. Application record is created
 5. Candidate views application status in dashboard
-

10. Security Considerations

- Login required for all candidate views
 - Duplicate applications prevented at database and logic level
 - Recruiter data is not exposed to candidates
-

11. Final Outcome of Phase 3

Phase 3 successfully delivers:

- Application model
- Candidate job listing
- Job application flow
- Duplicate protection
- Candidate dashboard with status tracking
- Clean and consistent Bootstrap UI

Phase 3 Outcome

Candidates can browse jobs, apply for jobs, and track application status.

This completes Phase 3 and prepares the system for Phase 4 (AI-based evaluation and ranking).